5. Implementation (Source Code & Execution)

5.1 Source Code

5.1.1 Structured & Well-Commented Code

The project follows a modular architecture that separates the presentation layer, business logic, data access layer, machine learning models, and AI chatbot into independent components. Each module has a single responsibility, improving maintainability and readability.

The backend is organized using FastAPI routers, services, repositories, models, and schemas. The frontend is implemented as a Streamlit application responsible only for user interaction and API communication. Machine learning scripts are separated from the backend, allowing independent model training and prediction.

Project structure:

```
├── backend/
│   ├── app/
│   │   ├── core/
│   │   │   ├── config.py
│   │   │   └── database.py
│   │   ├── models/
│   │   ├── repositories/
│   │   ├── routers/
│   │   ├── schemas/
│   │   ├── services/
│   │   └── main.py
│   └── requirements.txt
│
├── frontend/
│   └── app.py
│
├── ml/
│   ├── preprocessing.py
│   ├── trainer.py
│   ├── predictor.py
│   ├── model_manager.py
│   ├── train_models.py
│   └── generate_fake_history.py
│
├── data/
│   ├── raw/
│   └── processed/
│
├── docs/
│   ├── diagrams/
│   └── reports/
```

Each layer performs one responsibility:

- Router → receives HTTP requests
- Service → business logic
- Repository → database operations
- Models → ORM database models
- Schemas → API request/response validation

5.1.2 Coding Standards & Naming Conventions

The implementation follows Python's PEP-8 coding guidelines.

The following naming conventions are used:

Element	Convention	Example
Variables	snake_case	drug_price
Functions	snake_case	get_drug_details()
Classes	PascalCase	DrugRepository
Constants	UPPER_CASE	API_BASE_URL
Files	snake_case	prediction_service.py

Additional conventions include:

- Descriptive variable names
- Short reusable functions
- Separation of business logic
- Type hints where applicable
- RESTful endpoint naming

5.1.3 Modular Code & Reusability

The system is divided into reusable modules.

Backend modules:

Module	Responsibility
Routers	API endpoints
Services	Business logic
Repositories	Database access
Models	Database entities
Schemas	Validation
Core	Configuration

Machine Learning modules:

Module	Purpose
preprocessing.py	Dataset preparation
trainer.py	Prophet model training
predictor.py	Forecast generation
model_manager.py	Load trained models
generate_fake_history.py	Synthetic history generation

Frontend modules:

- Search interface
- Drug details page
- Prediction visualization
- AI assistant

This separation allows future replacement of one module without affecting the others.

5.1.4 Security & Error Handling

The system implements several security and reliability measures.

Input Validation

- FastAPI validates request parameters using Pydantic schemas.
- Invalid requests return HTTP 400 responses.

Database Protection

- SQLAlchemy ORM prevents SQL Injection attacks.
- Parameterized database queries are used.

API Error Handling

- Missing medicines return HTTP 404.
- Invalid prediction requests return informative error messages.
- Internal exceptions return HTTP 500 with appropriate logging.

Frontend Validation

- Empty search inputs are handled gracefully.
- API failures display user-friendly messages.
- Loading indicators improve user experience.

LLM Safety

The chatbot uses Retrieval-Augmented Generation (RAG), ensuring responses are generated only from the custom pharmaceutical knowledge base instead of unrestricted model knowledge, reducing hallucination risk.

5.2 Version Control & Collaboration

5.2.1 Version Control Repository

Git was used throughout development to track project progress and enable collaboration among team members.

Repository platform:

GitHub

The repository stores:

- Backend source code
- Frontend
- Machine learning models
- Documentation
- Reports

5.2.2 Branching Strategy

The project follows a Feature Branch workflow.

```
main
|
├── backend
├── frontend
├── machine-learning
├── data
└── documentation
```

Each feature was developed independently before merging into the main branch.

Advantages:

- Easier collaboration
- Reduced merge conflicts
- Independent feature testing

5.2.3 Commit History & Documentation

Meaningful commit messages were used.

Examples:

Add drug search endpoint

Implement Prophet prediction service

Integrate Streamlit frontend

Add RAG chatbot endpoint

Fix SQLite connection issue

Update API documentation

Improve prediction visualization

Each completed feature was committed independently.

5.2.4 CI/CD Integration

No automated CI/CD pipeline was implemented due to project scope.

Deployment and testing were performed manually.

Future improvements include:

- GitHub Actions
- Automated testing
- Automatic deployment
- Docker containers

5.3 Deployment & Execution

5.3.1 README File

The project README contains:

Installation

Clone repository

git clone <repository>

Install backend dependencies

pip install -r requirements.txt

Install frontend dependencies

pip install streamlit

System Requirements

Operating System

- Windows 10/11
- Linux
- macOS

Software

- Python 3.11+
- SQLite
- FastAPI
- Streamlit
- Transformers
- Prophet
- FAISS
- Sentence Transformers

Hardware

Minimum

- Dual-core CPU
- 8 GB RAM

Recommended

- Quad-core CPU
- 16 GB RAM
- NVIDIA GPU (optional)

Configuration

Configure:

database.py
config.py
Cloudflare Tunnel URL
External Drug API URL

Running the System

Backend

uvicorn app.main:app --reload --port 1000

Frontend

streamlit run app.py

LLM API

python chatbot_server.py

API Documentation

FastAPI automatically generates API documentation.

Swagger

http://localhost:1000/docs

Redoc

http://localhost:1000/redoc

5.3.2 Executable Files & Deployment

The current implementation is intended for local execution.

Deployment includes:

- FastAPI backend
- Streamlit frontend
- SQLite database
- Prophet prediction models
- Qwen RAG chatbot

Future deployment may use Docker or cloud hosting.

6. Testing & Quality Assurance

6.1 Test Cases & Test Plan

Test ID	Scenario	Expected Result	Status
TC-01	Search medicine	Matching medicines displayed	Pass
TC-02	Invalid medicine	No results message	Pass
TC-03	View drug details	Details displayed	Pass
TC-04	Generate prediction	Forecast graph shown	Pass
TC-05	Model unavailable	Warning displayed	Pass
TC-06	Ask chatbot medicine question	Accurate RAG answer	Pass
TC-07	Ask unrelated question	"Information unavailable" response	Pass
TC-08	External API offline	Cached data used if available	Pass
TC-09	Invalid API request	HTTP error returned	Pass
TC-10	Empty search	Empty state displayed	Pass

6.2 Automated Testing

Automated testing was not implemented due to project time constraints.

Testing consisted of:

- Manual endpoint testing
- Swagger testing
- Streamlit interface testing
- Prediction verification
- Chatbot verification

Future work may include:

- pytest
- unittest
- GitHub Actions

6.3 Bug Reports

Bug	Cause	Solution
SQLite connection issue	Incorrect configuration	Updated database configuration
Missing prediction model	Model not trained	Added model availability check
API timeout	External API delay	Added timeout handling
Empty chatbot responses	Retrieval failure	Improved FAISS indexing
Invalid search parameters	Missing validation	Added request validation